

Chapter 6

Missing Values

by Lorraine Gaudio

Version May 29, 2026

Contents

1	Overview	1
2	Open an R Notebook	2
3	Missing Values Workflow	3
3.1	Special Values	3
3.2	Detecting Missing Values	4
3.3	Locating Missing Values	4
3.4	Which Missing-Value Function Should I Use?	5
4	Nesting Function Calls	5
4.1	Locating Missing Values with Nesting	6
4.2	Summarizing “Missingness”	6
4.3	Imputation	6
5	Summary	9
6	Chapter Terms	9
7	Practice Space	10
7.1	Task 1	10
7.2	Task 2	10
7.3	Task 3	11
7.4	Task 4	11
7.5	Task 5	12
7.6	Task 6	12
7.7	Task 7	13
7.8	Task 8	13
7.9	Task 9	14

1 Overview

In this chapter, you will practice R habits that help prevent your analysis from being quietly wrong. Code can run without producing a useful or trustworthy result. This chapter focuses on checking assumptions, understanding function defaults, and handling missing values deliberately.

You will work with special values like `NA` and `NaN`, practice summary functions, detect and locate missing values, and compare two basic ways to handle missing data: removing missing values and replacing them

with a simple summary value. You will also practice nesting functions, which means putting one function inside another to solve multi-step problems.

In Chapter Six, you will practice how to:

- Recognize `NA` as missing information and `NaN` as an undefined numeric result.
- Describe how missing values can affect calculations in R.
- Recognize that many R functions have default arguments that affect results.
- Use `na.rm = TRUE` to tell summary functions to ignore missing values.
- Use `anyNA()` to check whether a vector contains missing values.
- Use `is.na()` and `which()` to identify where missing values are located.
- Use `sum(is.na(x))` and `mean(is.na(x))` to summarize how much data is missing.
- Recognize the difference between removing missing values and imputing missing values with a summary value.
- Read nested function calls from the inside out.
- Explain why compact code can be efficient but harder to read and debug.

2 Open an R Notebook

Before you follow along with Chapter 6, set yourself up the same way you did in Chapter 5. Your work will be readable and helpful for you to study for review.

Please open a new R Notebook in your course folder named `chapter6_notes.Rmd` and add a date to the YAML header. Use this document to take notes, practice the chapter content, and memo about what you are learning.

The **YAML header** is the setup block at the very top of an R Notebook. It tells R Markdown the document title, author, date, and output type.

1. Open RStudio → File → New File → R Notebook.
2. Save it in your course folder as: `chapter6_notes.Rmd`.
3. In your YAML header at the top:
 - Update the `title` to match this chapter.
 - Include a `date` (hard-coded or dynamic, your choice).
 - Add your name as the `author`.

For example, your YAML header might look like this:

```
---
title: "Chapter 6 Notes"
author: "[Your Name]"
date: "`r Sys.Date()`"
output: html_notebook
---
```

The date can be typed directly, such as “2026-05-20”, or it can be dynamic using `r Sys.Date()` so the document updates the date automatically when it renders.

4. Remove the default content.

5. Use headings as you work. At minimum, create section headings that match the chapter structure (e.g., `# Special Values`, `# Built-in Functions`, `# Missing Values Workflow`, `# Nesting`). This creates a readable outline.
6. Memo as you go.
 - Inside chunks: use `#` for code comments (otherwise R tries to run the text as code).
 - Outside chunks: write normal text without a `#`.
 - If you hit an error, write a one-sentence note about what you changed to fix it. Learned something new? Write a one-sentence note about that too.
 - Create these notes as a resource for you to review later.

Next, you'll focus on what NA means, how missing values “propagate” through calculations, and how to detect and locate missing values.

3 Missing Values Workflow

The `na.rm` argument in the `mean()` function has a default value of `FALSE`, meaning that if you don't specify it, the function will include NA values in its calculation.

3.1 Special Values

In R, **missing values** are represented by the **special value** symbol NA (which stands for “Not Available”).

R has several **special values** that represent non-standard data states. Two common ones are NA and NaN.

- **NA** (“Not Available”) signals *missing* information: the value simply was not recorded.
- **NaN** (“Not a Number”) signals an *undefined* numeric result, such as 0 divided by 0.

```
# Type this in your R Notebook
0 / 0      # Produces NaN - undefined arithmetic

NA         # Built-in constant representing a missing value
```

R prints NA and NaN in the Console so you can spot them quickly. Even though they look similar, they do not mean exactly the same thing.

- NA means the value is absent or unknown.
- NaN means the result is not a valid number, usually because the math operation is undefined.

We will mostly focus on NA in this chapter because missing values are common in real datasets. Missing values can occur in datasets because of incomplete data entries such as non-responses in surveys, or when joining two datasets. R *will* do the operation, but the result becomes NA because the value is unknown (e.g., `5 + NA` returns NA; that's not an error; that's nothing.). This is called **propagation**.

The SYNTAX (with `na.rm` default argument):

```
mean(x, na.rm = FALSE)
```

where `na.rm` is a logical argument that tells the function whether to remove NA values before calculating the mean.

Add NA value(s) to the cursed sequence and calculate the mean while handling missing values.

```
# Type this in your R Notebook
cursed_seq <- c(4, 8, 15, 16, 23, 42, NA) # includes a missing value
# cursed_seq <- c(cursed_seq, NA, NA)    # uncomment to add more missing values
```

```
mean(cursed_seq) # default is na.rm = FALSE + returns NA
mean(cursed_seq, na.rm = TRUE) # Change default to TRUE to ignore missing values
```

The first `mean()` call returns `NA` because the default behavior includes missing values in the calculation. The second call uses `na.rm = TRUE` to ignore the `NA` values, allowing the function to compute the mean of the remaining numbers.

In many summary functions, any `NA` in the vector causes the result to be `NA` unless you tell R to remove missing values first.

Function	What it does	Example
<code>sum()</code> , <code>prod()</code>	Sum/product	<code>sum(x, na.rm = TRUE)</code>
<code>mean()</code> , <code>median()</code>	Central tendency	<code>mean(x, na.rm = TRUE)</code>
<code>min()</code> , <code>max()</code> , <code>range()</code>	Extrema/range	<code>range(x, na.rm = TRUE)</code>
<code>sd()</code> , <code>var()</code>	Spread	<code>sd(x, na.rm = TRUE)</code>

3.2 Detecting Missing Values

A quick yes/no check prevents surprises later in your workflow.

- `anyNA(x)` returns `TRUE` if *any* element in `x` is `NA` or `NaN`.

The SYNTAX (basic):

```
anyNA(x)
```

where `x` is the vector you want to check for missing values.

Use `anyNA()` to check if there are any missing values in a vector.

```
# Type this in your R Notebook
Vector_NA <- c(3, 7, NA, 12) # Our sample data - one value is missing
anyNA(Vector_NA) # TRUE means "Something is missing!"
```

If was my data and I saw `FALSE`, I could relax; the vector has no gaps. Seeing `TRUE` tells me to investigate further.

3.3 Locating Missing Values

There are two useful functions to find the exact positions of missing values in a vector:

- `is.na(x)` returns a logical vector: `TRUE` wherever `x` is `NA` or `NaN`.
- `which(logical_vec)` converts `TRUE` positions into numeric indices (handy for slicing or replacing values).

Use `is.na()` and `which()` to locate missing values in `Vector_NA`.

```
# Type this in your R Notebook
is.na(Vector_NA) # Notice that it is TRUE at position 3
```

The `is.na()` function creates a logical vector indicating which elements are missing. The third element is `TRUE`, indicating that the third element in `Vector_NA` is `NA`.

Vectors can be quite long, so it's often more useful to get the numeric indices of missing values. Use `which()` to extract those indices.

```
# Type this in your R Notebook
logical_NA <- is.na(Vector_NA)
which(logical_NA) # Returns index 3 directly
```

The `is.na()` function creates a logical vector `logical_NA` indicating which elements are missing, while the `which()` function extracts the *indices* of those missing values (see **Positional Indexing**).

3.4 Which Missing-Value Function Should I Use?

Several functions help you work with missing values. They answer different questions.

Question	Function	What it returns
Are there any missing values?	<code>anyNA(x)</code>	One TRUE or FALSE answer
Which values are missing?	<code>is.na(x)</code>	A logical vector
What positions are missing?	<code>which(is.na(x))</code>	Numeric index positions
How many values are missing?	<code>sum(is.na(x))</code>	A count
What proportion is missing?	<code>mean(is.na(x))</code>	A proportion

The key is to choose the function that matches your question. If you only need a quick yes/no check, use `anyNA()`. If you need the exact positions, use `which(is.na())`.

4 Nesting Function Calls

Nesting function calls means putting one function call inside another function call. The output of the inner function becomes an argument to the outer function. This works because functions return values. R can immediately pass a returned value into the next function.

Use `sample()` to randomly select 5 numbers from the vector `travel_jumps`, then calculate the mean of those sampled numbers by nesting the `sample()` function inside the `mean()` function.

```
# Type this in your R Notebook
# create travel_jumps vector
travel_jumps <- seq(from = 1887, to = 2052, by = 33)

# Read nested code from the inside out
mean(sample(travel_jumps, size = 5, replace = TRUE)) # sample then average
```

R reads this as:

1. Run `sample(travel_jumps, size = 5, replace = TRUE)` first.
2. Return 5 sampled values.
3. Pass those 5 values into `mean()`.
4. Calculate the average.

Nesting is the simplest form of function composition. It builds a multi-step transformation by combining small, single-purpose functions.

```
# Nesting with three functions; creating the travel_jumps object
mean(sample(seq(from = 1887, to = 2052, by = 33), size = 5, replace = TRUE))
```

The nesting example with three functions shows that the sequence is first calculated, then sampled, and finally, the sample mean is computed.

Real work is rarely one step. Nesting is the base-R way to express that workflow without writing a ton of intermediate objects. However, nesting creates parentheses-heavy code that’s harder to read and debug. In later chapters, we will learn how to pipe multi-step workflows. For now, we’ll practice finding missing values with nesting functions.

4.1 Locating Missing Values with Nesting

Create a large vector with some missing values using `sample()` and `c()`.

```
# Type this in your R Notebook
set.seed(100)
LargeVec <- sample(c(1:10, rep(NA, 10)))
print(LargeVec)
```

We used the `c()` function in Chapter 2. Here we have nested `c()` inside of `sample()`. We also used the `rep()` function, which is covered in more detail in a later chapter.

Use `which()` and `is.na()` to find the positions of missing values in a large vector by nesting the `is.na()` function inside the `which()` function.

```
# Type this in your R Notebook
which(is.na(LargeVec))           # Where are the gaps?
```

The `is.na()` function creates a *logical* vector indicating which elements are missing, and the `which()` function determines the indices of those TRUE values. The numbers are the numeric indices (position) of the NA values.

4.2 Summarizing “Missingness”

As a researcher, knowing *how much* data is missing guides your next step.

A single NA might be harmless. If half of your data is missing, that might be big problem.

For example, if 1 out of 20 values is missing, then 5% of the vector is missing. If 10 out of 20 values are missing, then 50% of the vector is missing. Those situations should not be treated the same way. If and how you analyze this data depends on how much is missing.

Use `sum()` and `mean()` with `is.na()` to summarize missing values in a large vector.

```
# Type this in your R Notebook
sum(is.na(LargeVec))

mean(is.na(LargeVec))
```

The `sum(is.na(LargeVec))` expression counts the total number of missing values in `LargeVec` by summing the TRUE values from `is.na()`. The `mean(is.na(LargeVec))` expression calculates the proportion of missing values by taking the mean of the logical vector returned by `is.na()`, where TRUE is treated as 1 and FALSE as 0. Fifty percent missing values yield a mean of 0.5.

4.3 Imputation

Now that you can *detect* and *locate* gaps, you have to decide what to do about them. There is no universal “best” choice because each option makes assumptions.

In this chapter, we’ll use mean imputation as a practice method for learning replacement logic. It is not automatically the best choice in real research. Replacing missing values with the mean can make the data look more complete than it really is, and it can distort the distribution, especially the variance.

If you do not know why values are missing, be cautious.

- **Option 1:** Remove missing values (complete-case analysis). Simple and common, but you may drop a lot of data (and potentially bias results).
- **Option 2:** Replace missing values with a summary value (simple imputation). Keeps the vector length the same, but it can distort the distribution (especially the variance).

In this chapter we're practicing on a single vector, so removing missing values often looks "safer." In real analysis, though, values live in rows of a dataset (one row = one person/observation) with multiple columns. If you remove rows because one column is missing, you also throw away valid information in other columns. That can:

- shrink your sample size (less stable results),
- change which groups are represented (bias),
- break analyses that require the full row structure (plots, models, group summaries).

Imagine a class dataset with `student_id`, `major`, `exam1`, and `exam2`. If `exam2` is missing for some students, removing those students from the analysis (rows) means we'd also lose their `exam1` scores and their `major`. If our goal is "average Exam 1 by major," dropping rows because Exam 2 is missing can silently change the story. This is especially true if missing Exam 2 happens more in one major than another.

The bottom line is that there is no one-size-fits-all solution. If you only need a quick summary of one variable, removing NAs is often reasonable. If you need to keep rows aligned across variables, or you are comparing groups, you may consider imputation.

Whatever you choose, you need to document that choice and explain why it made sense for your goal. This is one reason why writing memos around your analysis is so important.

Next, you'll practice both strategies on a vector so you can see the trade-off clearly. Later, you'll apply the same decision logic to multi-column datasets.

4.3.1 Option 1

Remove missing values

Use indexing with `!is.na()` to keep only the observed (non-missing) values in `LargeVec`.

```
# Type this in your R Notebook
clean_vec <- LargeVec[!is.na(LargeVec)] # Keeps only observed values
clean_vec
```

`is.na(LargeVec)` returns a logical vector (TRUE for missing, FALSE for present). The `!` flips it (so TRUE means "keep this value"). The brackets `[...]` then use that logical vector to select only the elements you want.

4.3.2 Option 2

Impute missing values (replace NA with the mean)

Impute means "fill in missing values." A common simple imputation method is to replace missing values with the mean of the observed values.

Make a copy of `LargeVec`, then replace only the missing values with the mean of the observed values.

```
# Type this in your R Notebook
imputed <- LargeVec      # Make a copy to preserve original
```

The reason we make a copy is that we want to keep the original vector with its missing values intact for comparison. This way, we can see the effect of imputation without losing the original data.

Tip: Once we make a copy, we use that copy name everywhere in the imputation line. This new version of the vector is called `imputed`, and it is the version we will use next. `imputed` is the vector's *name*. The name is descriptive of what the difference will be between it and the original `LargeVec` once we have replaced only the missing values with the mean of the observed values. It is good practice to name objects in meaningful ways.

We're doing three actions on the same object (`imputed`):

(1) identify missing positions, (2) compute a replacement value, and (3) write the replacements back.

```
# Type this in your R Notebook
imputed[is.na(imputed)] <- mean(imputed, na.rm = TRUE)
imputed
```

`is.na(imputed)` identifies exactly where the gaps are, and `imputed[...] <- ...` replaces only those positions. The `na.rm = TRUE` argument is required here. Without it, `mean(imputed)` would return NA because the input contains missing values.

The object we are editing is on the left side: `imputed[is.na(imputed)]`.

The object we are using to find the missing positions is inside brackets: `is.na(imputed)`.

The general replacement pattern is:

```
object[positions_to_replace] <- replacement_value
```

Notice: You might think, “The missing values came from `LargeVec`, so I should check `LargeVec`.” But the rule is to use the same object we're editing, or we risk indexing the wrong thing later. The object we are summarizing to compute the replacement value (mean): `mean(imputed, na.rm = TRUE)`

Notice: You might think, “mean of observed values” means “mean of the original” and want to use `LargeVec`. But `imputed` currently equals `LargeVec`, so the mean is identical either way at that moment. As soon as you start changing `imputed`, mixing names can cause logic bugs. You might replace values in one object using positions or summaries computed from a different object.

In summary, wherever an element in `imputed` is missing, we replace those spots with the mean of `imputed` while ignoring missing values so it can calculate the mean.

Confirm your result by comparing `sum(is.na(LargeVec))` versus `sum(is.na(imputed))`.

```
#
sum(is.na(LargeVec))      # Original vector - should print number 10
sum(is.na(imputed))      # Imputed vector - should print 0
```

The first `sum(is.na(LargeVec))` call counts the number of missing values in the original vector, which should be greater than 1. The second `sum(is.na(imputed))` call counts the missing values in the imputed vector, which should be 0, confirming that all missing values have been replaced.

5 Summary

In this chapter, you strengthened your R workflow by learning how to recognize and handle special values in R. You practiced the difference between `NA`, which means information is missing, and `NaN`, which means a numeric result is undefined. You also learned that missing values can quietly affect your calculations because many R functions return `NA` unless you tell them how to handle missingness.

You built a practical missing-values workflow. You practiced using `anyNA()` to check whether missing values exist, `is.na()` to identify which values are missing, `which()` to locate their positions, `sum(is.na(x))` to count missing values, and `mean(is.na(x))` to calculate the proportion missing. You also practiced using `na.rm = TRUE` so summary functions can calculate results while ignoring missing values.

Finally, you practiced two basic strategies for handling missing values: removing missing values with logical indexing and replacing missing values with a simple imputed value. You also practiced nesting function calls so that one function’s result can be passed directly into another function. Your main takeaway is that strong R work depends on checking assumptions, understanding defaults, documenting your choices, and making sure your code is not only running, but producing results you can trust.

6 Chapter Terms

Impute: To replace missing or unusable values (e.g., `NA`) with substituted values—often a plausible estimate based on the observed data (such as mean/median imputation or model-based predictions)—so the dataset can be analyzed without dropping incomplete cases. Because imputed values are estimates, you should document what method you used and why.

Missing Values: Data values that are unknown or not recorded; in R they are represented by `NA` (“Not Available”). `NA` is a special missing-value indicator (with typed versions like `NA_integer_`, `NA_real_`, and `NA_character_`), and many calculations will return `NA` if any missing values are present unless you explicitly remove them (often via `na.rm = TRUE`, which defaults to `FALSE` in functions like `mean()`).

NA: A special constant that marks a missing value (“Not Available”)—meaning the data point is unknown or not recorded. R also provides typed missing values like `NA_integer_`, `NA_real_`, `NA_character_`, and `NA_complex_` to match the underlying data type.

NaN: A special numeric constant (“Not a Number”) that represents an undefined arithmetic result in floating-point math (for example, `0/0` or other invalid operations). `NaN` is non-finite (not a regular number).

Nesting Function Calls: Writing one function call inside another so the inner function runs first and its returned value becomes an argument to the outer function (for example, `mean(sample(x, 5))`). This works because a function call can appear anywhere an expression is allowed, and functions return a value that can be passed directly into the next call.

Positional Indexing: Indexing a vector using numeric positions (for example, `x[1]` for the first element or `x[c(1, 3)]` for the first and third). Positions are based on the current order of the vector.

Propagation: R’s default behavior where an unknown value “spreads” through a computation. If an expression includes `NA`, the result will usually be `NA` (e.g., `5 + NA` returns `NA`) because the outcome can’t be determined without the missing information. This is often called **NA propagation**, and it’s why many summary functions return `NA` unless you explicitly remove missing values (e.g., `na.rm = TRUE`).

Special Values (`NA`, `NaN`, `Inf`, `-Inf`, `NULL`): Reserved constants in R that represent non-standard values: `NA` for missing data, `NaN` for undefined numeric results, `Inf`/`-Inf` for positive/negative infinity, and `NULL` for “no value” (an empty/null object). In practice, use `is.na()` to detect `NA` and `NaN`, `is.nan()` to detect `NaN` specifically (not `NA`), `is.infinite()/is.finite()` to handle infinities and other non-finite values, and `is.null()` to test for `NULL`.

7 Practice Space

Fill in the blanks and answer the short prompts in your R Notebook. This Practice Space helps you check whether you can organize a notebook, read an R help page, use a function argument, and knit your work into an HTML report.

Before you begin:

1. Create a new R Notebook in RStudio.
2. Save it in your course folder as `chapter6_practice.Rmd`.
3. Complete each task in order.
4. Add short memo notes throughout your notebook. Your memo notes should explain what your code does, what output you expected, what surprised or confused you, and what you would check next if something did not work.

In this Practice Space, you will see two kinds of memo prompts:

- **Add Comments** asks you to explain what the code or output shows.
- **Reflect** asks you to explain your thinking process, what confused you, what changed in your understanding, or what strategy you would use next.

These are there to help you practice the habit of writing memos as you work. **In each task, you can choose whether to answer one, two, or all memo prompts.**

If you encounter an error, document your **tenacity** by briefly noting what happened, what you tried, and how you fixed it or what still confuses you. Document how you troubleshooted the error and what resource(s) you used to help you resolve the issue (**transparency**). Show **courage** by sharing your thinking process, even if it includes mistakes or confusion.

You may quote AI output only when you clearly mark it as quoted text, either with quotation marks or a block quote using `>`. Then hand-type your own explanation of why you are including that quote.

Before you finish, clear your Environment and run the notebook from top to bottom. Then knit to HTML to check that the notebook works in order.

7.1 Task 1

Help-file scavenger hunt (required vs default)

1. Use `?sample` to learn about the `sample()` function.
 - Focus first on the `sample()` Usage line, not the `sample.int()` line.
2. In one sentence, explain how you can tell required vs default arguments from the **Usage** line.
3. Write the Usage lines exactly as they appear (copy/paste is fine), then label:
 - TWO required arguments
 - TWO default arguments for `sample()` exactly as they appear in the Usage section. How you can tell which arguments are required?

Add Comments: Explain how the Usage line helped you decide which arguments were required and which had default values.

Reflect: What part of the help page was easiest to interpret? What part was still confusing or too technical?

7.2 Task 2

Build a random data set

Create scores as 28 values. That means you will create 25 random integers from 60 through 100, then add 3 missing values (NA) at the end.

1. First, use `seq()` to create the possible score values from 60 through 100. Store that sequence as `pool`.

Tip: For this task, practice using `seq()` with separate inputs for the starting value and ending value. Do not use the `:` shortcut inside `seq()` here. If you are unsure what arguments `seq()` expects, look at `?seq` and focus on the Usage line.

2. Then use `sample()` to draw 25 values from `pool`, with replacement. Store those sampled values as `scores_numbers`.
3. Finally, use `c()` to add three NA values to the end of `scores_numbers`. Store the full vector as `scores`.

```
# Type this in your R Notebook, Fill in the blanks
set.seed(1)

pool <- ____ # use seq() to create 60 through 100
scores_numbers <- ____ # use sample() to draw 25 values (with replacement)
scores <- c(____, NA, NA, NA) # add 3 `NA` to the end of `scores_numbers` using `c()`

# Check the length of scores
____(scores)
```

Add Comments: Explain why `scores` has 28 elements even though `sample()` only drew 25 values.

Reflect: Which part of the code built the possible values, which part randomly selected values, and which part added missing values? How did checking `length(scores)` confirm your thinking?

7.3 Task 3

Handle missing data

Using `scores` from Task 2:

1. Count missing values. Store as `n_missing`.
2. Compute the mean with defaults. Store as `average_raw`.
3. Compute the mean excluding NAs and store it as `average_score`.
4. Explain why `average_raw` behaves the way it does (use the word “default”).

```
n_missing <- ____ # hint: is.na() + sum()

# Check the number of missing values
____ # should print 3

# Type this in your R Notebook, Fill in the blanks
average_raw <- mean(____) # what happens?

# Type this in your R Notebook, Fill in the blanks
average_score <- mean(____, ____ )
# print the mean
____
```

Add Comments: Explain why `average_raw` returns NA. Use the word **default** in your explanation.

Reflect: Before you ran the code, did you expect R to ignore the missing values or return NA? How did this task change your understanding of how R handles missing values?

7.4 Task 4

Locate missing values

Using `scores` from Task 2:

1. Use `is.na()` to create a logical vector showing which elements of `scores` are missing; store it as `scores_is_missing`.
2. Nest `is.na()` inside `which()` to find the positions (indices) of the missing values; store as `missing_positions`.
3. Verify your result

```
# Type this in your R Notebook, Fill in the blanks
scores_is_missing <- is.na(____)

missing_positions <- which(____(____))

# Check
print(____)      # should print a logical vector
print(____)      # should print indices of missing values
length(____)     # should print 3
```

Add Comments: Explain the difference between the output of `is.na(scores)` and the output of `which(is.na(scores))`.

Reflect: Which output would be more useful if you only wanted to know whether values were missing? Which output would be more useful if you needed to replace or inspect the exact missing positions?

7.5 Task 5

Nesting functions

Compute the mean of 5 random draws from `scores` in one line, and make sure it still works even if the draw includes NA. **Tip:** Read the nested code from the inside out. `sample()` should happen first. Then `mean()` should summarize the sampled values. Use `na.rm = TRUE` inside `mean()` so the mean can still be calculated if the sample includes NA.

```
# Type this in your R Notebook, Fill in the blanks
set.seed(21)
mean_of_five <- ____
# Check
mean_of_five
```

Add Comments: Explain which function runs first in your nested code and why `na.rm = TRUE` is needed.

Reflect: Was the nested version easier or harder to understand than writing the steps separately? If you were debugging this code, would you keep it nested or break it into smaller objects? Explain why.

7.6 Task 6

Quick time-stamp

Use `Sys.Date()` and `Sys.time()` to create two separate objects, `today_date` and `now_time`.

```
# Type this in your R Notebook, Fill in the blanks
today_date <- ____
now_time <- ____
```

Add Comments: Explain the difference between the output of `Sys.Date()` and `Sys.time()`.

Reflect: Why might a researcher want a report to include the date or time when the code was run? How could a time-stamp support reproducibility or transparency?

7.7 Task 7

Handling Missing Values

Using scores from Task 2:

Part A — Remove NA values (“complete-case” for a vector)

1. Create `scores_clean` by keeping only the observed (non-missing) values in `scores`.
2. Verify that `scores_clean` has 3 fewer values than `scores`.

Type this in your R Notebook, Fill in the blanks

```
scores_clean <- scores[!_____ (_____)]
```

Checks

```
length(scores)           # should print 28
length(scores_clean)     # should print 25
```

Add Comments: Explain what `is.na(scores)` identifies, what `!is.na(scores)` changes, and how bracket indexing uses that result to create `scores_clean`.

Reflect: Why is removing missing values simple for one vector but potentially risky in a larger dataset with many columns?

Part B — Impute: replace NA with the mean of present values

1. Make a copy of `scores` named `scores_imputed` (so you don’t overwrite the original).
2. Replace the missing values in `scores_imputed` with the mean of the observed values.
3. Verify there are 0 missing values after imputation.

Type this in your R Notebook, Fill in the blanks

```
scores_imputed <- _____
```

```
scores_imputed[_____] <- mean(_____, na.rm = _____)
```

Checks

```
sum(is.na(scores_imputed)) # should print 0
mean(scores_imputed)       # should print a number (not NA)
```

Add Comments: Explain why you made a copy named `scores_imputed` before replacing missing values.

Reflect: What assumption are you making when you replace missing values with the mean? Why should this choice be documented instead of treated as automatic?

7.8 Task 8

Reflection

In one sentence, explain the difference between a *function object* such as `mean` and a *function call* such as `mean()`.

Reflect: What kind of mistake might happen if you type a function name without parentheses when you meant to run the function?

7.9 Task 9

Step 1: Sweep your Environment (start clean).

Click the **broom icon** in the Environment pane to clear objects.

Step 2: Run everything top-to-bottom.

Use **Run** → **Run All** to execute every chunk in order.

Step 3: Save and submit.

After your notebook runs without errors, save your notebook (`.Rmd`) and knit to HTML (`.HTML`) for submission to Canvas, as directed.

Tip: After you knit, look in your course folder for two files with the same name. One file ends in `.Rmd` and one ends in `.html`. Upload the `.html` file. You can check which is which because one opens in R and the other usually opens in a web browser.

Final Reflection: What was one place where your code did not behave the way you expected? What did you check or change before submitting? How much time did you spend completing the Practice Space? Did you complete the Guided Activity first?